

20260805

gLike current optimization function

- Current optimization approach is [here](#)
 - Best described as a greedy coordinate descent.
 - Each epoch/generation of the iterator, we evaluate each param independently, holding all other params fixed
 - Whichever positive or negative stepsize for a given coordinate best improves likelihood --> selected
 - Every accepted move across the surface is guaranteed to increase objective function
 - There is an adaptive component by which whenever movement in one direction improves the objective function, we take bigger steps.
 - If movement in either direction does not improve the objective, we take smaller steps along that coordinate
 - This is kind of like a Powell method
 - Notably, derivative and gradient free, single-param (or single direction) optimization, and can work well for convex multi-dimensional surfaces at the cost of accuracy
 - Biggest area of improvement would be to traverse multiple parameters simultaneously in an optimal direction (off the top of my head)

Implementation: General benchmarking pipeline

- [Covariance matrix adaptation evolution strategy \(CMA-ES\)](#)
- Start by simulating ARG with equidistant trees

```
usage: simulate_and_fit_ARG.py [-h] [--N_SAMPLES N_SAMPLES] [--SEQUENCE_LENGTH SEQUENCE_LENGTH]
                             [--RECOMBINATION_RATE RECOMBINATION_RATE] [--SEED SEED]
                             [--NUM_TREES NUM_TREES] [--OPTIMIZER OPTIMIZER]
                             outprefix

Simulate a given number of equally distant trees using the true demographic parameters of a Native
Hawaiian Ancestral Recombination Graph (ARG). We then fit those trees to the demographic model of the NH
scenario.

positional arguments:
  outprefix            The file prefix for the output files.

optional arguments:
  -h, --help            show this help message and exit
  --N_SAMPLES N_SAMPLES
                        Number of samples to be simulated. (default: 1000.0)
  --SEQUENCE_LENGTH SEQUENCE_LENGTH
                        Length of sequence to be simulated. (default: 30000000.0)
  --RECOMBINATION_RATE RECOMBINATION_RATE
                        Rate of recombination. (default: 1e-08)
  --SEED SEED           Random seed. (default: 1)
  --NUM_TREES NUM_TREES
                        Number of equally distant trees to simulate. (default: 20)
  --OPTIMIZER OPTIMIZER
                        Optimization strategy for fitting ARGs. (default: maximize)
```

- ○ Number of samples, sequence length, recombination rate, random seed, number of trees, different optimizers
- Confirm that 20 different equidistant trees are simulated:

```
(glike) bash-4.4$ ls
NH_ARG_0.trees  NH_ARG_11.trees  NH_ARG_14.trees  NH_ARG_17.trees  NH_ARG_2.trees  NH_ARG_5.trees  NH_ARG_8.trees  simulate_ARG.log
NH_ARG_1.trees  NH_ARG_12.trees  NH_ARG_15.trees  NH_ARG_18.trees  NH_ARG_3.trees  NH_ARG_6.trees  NH_ARG_9.trees
NH_ARG_10.trees NH_ARG_13.trees  NH_ARG_16.trees  NH_ARG_19.trees  NH_ARG_4.trees  NH_ARG_7.trees  NH_positions.json
```

- Confirm we can compute the true NH demography from the tutorial

```

      admix
ns    0.000006
grs   0.078000

[phase from 19.0 to 411.0]
      afr    eur    asia    pol
admix  0.0    0.198  0.334  0.468
      afr    eur    asia    pol
ns     0.0001  0.000075  0.00004  0.000064
grs    0.0000  0.000000  0.00000  0.000000

[phase from 411.0 to 1040.0]
      afr    eur    asia
afr     1      0      0
eur     0      1      0
asia    0      0      1
pol     0      0      1
      afr    eur    asia
ns     0.0001  0.000075  0.00037
grs    0.0000  0.000000  0.00000

[phase from 1040.0 to 2004.0]
      afr    eur
afr     1      0
eur     0      1
asia    0      1
      afr    eur
ns     0.0001  0.000405
grs    0.0000  0.000000

[phase from 2004.0 to inf]
      A
afr    1
eur    1
      A
ns     0.000375
grs    0.000000

```

```

2026-08-05 10:17:01.922 - INFO - Starting glike optimization.
FAILED PARAMETERS:
{'r1': 0.2640180443519721, 'r2': 0.22371108638370388, 'r3': 0.25097234547897285, 'N_admix': 9999.972311368547, 'N_afr': 10000.033236344045,
'N_eur': 9999.899305853232, 'N_asia': 9999.985468982719, 'N_pol': 9999.966888809295, 'N_aa': 10000.014971271925, 'N_ooa': 10000.036522233246,
'N_anc': 9999.98180777261, 'gr': 0.16161376904028268, 't1': 9.49994035931019, 't2': 96.42514090359553, 't3': 994.9633989437352, 't4': 1967.1460048309123}
Cannot initialize phase: t_end should be a number!
Traceback (most recent call last):
  File "simulate_and_fit_ARG.py", line 238, in <module>
    SimulateARG().main()
  File "simulate_and_fit_ARG.py", line 222, in main
    x, logp = estimate.maximize_CMA_ES(fun, x0, bounds = bounds)
  File "/home1/mahj/glike/optimize/opt_scripts/../glike/estimate.py", line 229, in maximize_CMA_ES
    Y = [objective(x) for x in X]
  File "/home1/mahj/glike/optimize/opt_scripts/../glike/estimate.py", line 229, in <listcomp>
    Y = [objective(x) for x in X]
  File "/home1/mahj/glike/optimize/opt_scripts/../glike/estimate.py", line 202, in objective
    y = fun(**params)
  File "simulate_and_fit_ARG.py", line 203, in fun
    demo = glike.native_hawaiians_demo(t1, t2, t3, t4, r1, r2, r3,
  File "/home1/mahj/glike/glike/models.py", line 250, in native_hawaiians_demo
    demo.add_phase(Phase(0, t1, [1/N_admix * math.exp(-t1 * gr)], [gr], populations = ["admix"]))
  File "/home1/mahj/glike/glike/glike.py", line 66, in __init__
    raise Exception("Cannot initialize phase: t_end should be a number!")
Exception: Cannot initialize phase: t_end should be a number!

```

- The primary issue I'm running into with this implementation is the requirement for boundaries in CMA-ES.
 - Specifically, CMA-ES requires finite boundaries, which poses an issue in the case of derived population times in ARGs, e.g., $t_1 < t_2 < t_3 < t_4$
 - I'm using a workaround in which we can encode input parameters (similar to a log-transform) from $t_i \rightarrow dt_i$, where:
 - $dt_1 = t_1$

- $dt2 = t1 + t2$
 - $dt3 = t1 + t2 + t3$
 - $dt4 = t1 + t2 + t3 + t4$
 - We then use $dt1$, $dt2$, $dt3$, $dt4$ all less than 1 which satisfies both the CMA-ES constraints and the biological constraints of the ARG
- I'm having some trouble actually implementing this transform

TODO

- In the future look to put `gvcf` files onto physical hard drive and cold storage
- Figure out how to release data on `dbgap` (mostly paperwork), Terra --> `dbgap`